Introduzione a Java

Indice

Introduzione

Caratteristiche principali del linguaggio di programmazione

Variabili e tipi di dato

Ingresso e uscita

Operatori

Istruzioni condizionali

Cicli

Array

Metodi (funzioni)

Un algoritmo completo: il valore massimo in un array

Operatori bit a bit

`Java`, `C++` e `Python` a confronto

Introduzione

Introduzione: un terzo linguaggio di programmazione

- Nei capitoli «Introduzione a `C++`» e «Introduzione a `Python`» abbiamo tradotto lo pseudocodice dei nostri algoritmi in un linguaggio di programmazione **compilato** e in uno **interpretato**
- In questo capitolo introduciamo un terzo linguaggio di programmazione, `Java`, che sta **a metà strada**: i suoi programmi sono compilati in un codice intermedio, che viene poi eseguito da una macchina virtuale
- La sua sintassi è molto vicina a quella di `C++`, ma ogni programma è organizzato in **classi**

Il percorso del capitolo (1/2)

Il capitolo segue lo stesso percorso dei due capitoli precedenti, in modo che i tre linguaggi possano essere confrontati sezione per sezione:

- le caratteristiche principali del linguaggio e la struttura di un programma
- variabili, tipi di dato, ingresso e uscita
- operatori, istruzioni condizionali e cicli
- gli array, e le liste dinamiche della classe `ArrayList`

Il percorso del capitolo (2/2)

- i metodi (il nome che `Java` dà alle funzioni), i loro parametri e i loro valori di ritorno
- un algoritmo completo, la ricerca del valore massimo in un array, scritto in pseudocodice e in `Java`
- gli operatori bit a bit e una tabella riassuntiva che confronta `Java` con `C++` e `Python`

Ogni frammento di codice del capitolo è stato compilato con il compilatore `javac` ed eseguito con la macchina virtuale `java` (`OpenJDK 25`): gli output riportati negli esempi sono quelli stampati dai programmi.

Indici: pseudocodice e Java

- Nello pseudocodice le componenti di un array $a[1..n]$ sono indicizzate da 1 a n
- In `Java`, come in `C++` e in `Python`, gli elementi di un array di dimensione n sono indicizzati da 0 a $n-1$
- La componente $a[i]$ dello pseudocodice corrisponde all'elemento `a[i-1]` del codice `Java`

Riepilogo

- `Java` è il terzo linguaggio del corso, dopo `C++` (compilato) e `Python` (interpretato): è compilato in un codice intermedio eseguito da una macchina virtuale
- La sua sintassi è vicina a quella di `C++`, e ogni programma è organizzato in classi
- La componente $a[i]$ dello pseudocodice è l'elemento `a[i-1]` in `Java`

Caratteristiche principali del linguaggio di programmazione

Introduzione: caratteristiche principali del linguaggio di programmazione

- In questa sezione descriviamo le caratteristiche principali di `Java`: come viene eseguito un programma, i tipi di dato, le classi e i metodi
- Le confrontiamo con `C++`, con cui `Java` ha in comune gran parte della sintassi
- Poi osserviamo lo scheletro che ogni programma `Java` ha in comune

Compilato e interpretato

- `Java` è un linguaggio **compilato e interpretato**:
  - il codice sorgente (file `.java`) viene prima compilato in **bytecode** (file `.class`) dal compilatore `Java javac`
  - poi il bytecode viene eseguito dalla **Java Virtual Machine** (JVM)

- Questo rende i programmi `Java` **indipendenti dalla piattaforma**: lo stesso bytecode funziona su qualunque macchina su cui sia installata una JVM

«scrivi una volta, esegui ovunque»

Tipi, classi e metodi

- `Java` è un linguaggio **fortemente e staticamente tipizzato**: ogni variabile deve essere dichiarata con il suo tipo di dato prima dell'uso, e il suo tipo di dato non può cambiare durante l'esecuzione

- `Java` è **orientato agli oggetti**: ogni programma è organizzato in **classi**. Il punto di ingresso di ogni programma `Java` è il metodo `main` di una classe

- `Java` supporta i **metodi** (il termine `Java` per le funzioni), che permettono un approccio modulare e riutilizzabile alla programmazione

Blocchi, istruzioni e commenti

- Come in C++, la struttura a blocchi del codice è definita dalle **parentesi graffe** {}, e ogni istruzione deve terminare con il simbolo riservato ";"
- Come in C++, un commento su una sola riga inizia con il simbolo riservato "//"
- I commenti su più righe sono racchiusi fra i simboli riservati "/*" e "*/"

Esempio

Commenti in `Java`

Questo è un esempio di commento su una sola riga:

```
1 // Questa è una riga di commento
```

Questo è un esempio di commento su più righe:

```
1 /* Questo è un
2    commento su più righe */
```

La struttura di un programma

- Prima di descrivere le singole istruzioni, osserviamo lo **scheletro** che ogni programma ha in comune
- Un programma `Java` è costituito da una o più **classi**
- L'esecuzione parte dal metodo

  ```
  public static void main(String[] args)
  ```

  della classe che viene lanciata

Esempio (1/2)

Lo scheletro di un programma

```
1 public class HelloWorld {
2
3     public static void main(String[] args) {
4         // Qui si scrivono le istruzioni del programma
5         System.out.println("Ciao, mondo!");
6     }
7 }
```

Una classe dichiarata `public` deve essere salvata in un file con lo stesso nome della classe: qui `HelloWorld.java`.

Esempio (2/2)

Il programma si compila e si esegue con i comandi

```
javac HelloWorld.java
java HelloWorld
```

- Il primo comando produce il file di bytecode `HelloWorld.class`
- Il secondo lo esegue (si noti che l'estensione non si scrive)

L'output è

```
Ciao, mondo!
```

Frammenti di codice

- Negli esempi seguenti scriviamo solo le istruzioni che contano
- A meno che non contengano un programma completo, i frammenti di codice vanno inseriti fra le parentesi graffe di `main`

Istruzioni, metodi e classi

Ogni istruzione `Java` appartiene a un metodo,
e ogni metodo appartiene a una classe.

Ogni istruzione termina con “;”.

Riepilogo

Il linguaggio:

- compilato in bytecode da `javac`, eseguito dalla JVM: indipendente dalla piattaforma
- fortemente e staticamente tipizzato, orientato agli oggetti, organizzato in classi e metodi
- parentesi graffe, "`;`" e commenti come in `C++`

La struttura di un programma:

- l'esecuzione parte da `public static void main(String[] args)`
- una classe `public` si salva in un file con il suo nome; `javac` compila, `java` esegue

Variabili e tipi di dato

Introduzione: variabili e tipi di dato

- Il codice sorgente opera su **variabili**, e ogni variabile ha un **tipo di dato**
- In questa sezione vediamo i tipi di dato fondamentali di `Java` e come si dichiarano e si assegnano le variabili
- Vediamo anche che cosa succede quando un intero non entra nel suo tipo di dato

I tipi di dato fondamentali (1/2)

I quattro tipi di dato fondamentali di `Java` sono:

1. Il tipo di dato "`int`" rappresenta un intero memorizzato con 32 bit: un numero intero da

$$-2^{31} = -2147483648 \quad \text{a} \quad 2^{31} - 1 = 2147483647$$

   Per interi più grandi si usa il tipo di dato `long` (64 bit)

2. Il tipo di dato "`boolean`" rappresenta un valore booleano: `true` o `false` (minuscolo)

I tipi di dato fondamentali (2/2)

3. Il tipo di dato "`double`" rappresenta un numero in virgola mobile a precisione doppia (64 bit), come in `C++`. Il tipo di dato `float` ha precisione singola, e i suoi valori si scrivono con il suffisso `f` (ad esempio `3.14f`)

4. Il tipo di dato "`char`" rappresenta un singolo carattere, racchiuso fra apici singoli (ad esempio `'A'`, `'7'`)

- Il tipo di dato "`String`" (con la `S` maiuscola) rappresenta una sequenza di caratteri racchiusa fra virgolette doppie, ed è il modo standard di trattare il testo in `Java`

Variabili

- Una variabile funziona come un contenitore che memorizza un valore di un tipo di dato specifico per un uso successivo
- Alle variabili si assegnano valori con l'operatore di assegnamento "=". Quando si assegna un nuovo valore, il valore precedente viene sovrascritto
- Il compilatore controlla i tipi di dato: assegnare un valore di tipo sbagliato (per esempio la stringa `"cinque"` a una variabile di tipo `int`) o usare una variabile che non ha ricevuto un valore è un **errore di compilazione**, e il programma non viene prodotto

Dichiarazione prima dell'uso

Le variabili devono essere dichiarate con il loro tipo di dato prima dell'uso, come in C++.

Esempio (1/2)

Dichiarazioni e assegnamenti di variabili

```
1  int age = 25;                    // Variabile intera
2  boolean isRaining = false;       // Variabile booleana
3  double pi = 3.14159;             // Virgola mobile a precisione doppia
4  float small = 3.14f;             // Virgola mobile a precisione singola
5  double large = 123456.789;       // Virgola mobile a precisione doppia
6  char letter = 'A';               // Variabile carattere
7  String name = "Alice";           // Variabile stringa
8
9  age = 26; // Il valore precedente 25 viene sovrascritto
10 System.out.println(age + " " + isRaining + " " + pi + " " + small);
11 System.out.println(large + " " + letter + " " + name);
```

Esempio (2/2)

Output:

```
26 false 3.14159 3.14
123456.789 A Alice
```

- A differenza di C++, il valore booleano è visualizzato come `false`
- Il numero `123456.789` è visualizzato con tutte le sue cifre

Esempio (1/2)

I limiti del tipo di dato `int`

```
1 int big = Integer.MAX_VALUE; // Il più grande int
2 System.out.println(big);
3 System.out.println(big + 1); // Overflow: il risultato non entra in 32
      bit
4
5 long bigger = (long) big + 1; // Calcolato con 64 bit
6 System.out.println(bigger);
```

Output:

```
2147483647
-2147483648
2147483648
```

Esempio (2/2)

- Quando il risultato di un'operazione fra valori `int` non entra in 32 bit, «riparte» dall'altro estremo dell'intervallo, senza alcun messaggio di errore
- Qui $2^{31} - 1$ più 1 viene stampato come $-2^{31} = -2147483648$
- Gli interi di `Python` non vanno mai in overflow

Riepilogo

Tipi di dato:

- `int` (32 bit) e `long` (64 bit), `boolean`, `double` e `float`, `char`, e `String` per il testo

Variabili:

- dichiarate con il loro tipo di dato prima dell'uso, assegnate con "="
- il compilatore controlla i tipi di dato: un tipo sbagliato è un errore di compilazione
- un `int` che non entra in 32 bit riparte dall'altro estremo, senza messaggi di errore

Ingresso e uscita

Introduzione: ingresso e uscita

- Un programma deve mostrare i suoi risultati e, spesso, leggere i suoi dati
- In questa sezione vediamo come stampare valori sullo schermo, anche con un testo formattato
- Poi vediamo come leggere l'ingresso da tastiera con la classe `Scanner`

Stampe sullo schermo

- L'oggetto `System.out` (l'uscita standard, fornita dalla classe `System`) ha i metodi che visualizzano valori sullo schermo
- I più comuni sono:
  - `System.out.println()`: stampa un valore seguito da un a capo
  - `System.out.print()`: stampa un valore senza l'a capo

Concatenazione e stampa formattata

- Valori e stringhe sono **concatenati** con l'operatore + dentro un'istruzione di stampa: quando uno dei due operandi di + è una `String`, l'altro viene convertito in testo
- L'espressione è valutata da sinistra a destra, quindi un'operazione aritmetica dentro una concatenazione deve essere racchiusa fra parentesi
- Il metodo `System.out.printf()` stampa un testo formattato: nella prima stringa,
  - `%d` è sostituito da un intero, `%s` da una stringa
  - `%.kf` da un numero in virgola mobile con esattamente k cifre dopo la virgola decimale
  - `%n` da un a capo

Esempio (1/2)

Stampe sullo schermo

```
1  int n = 42;
2  System.out.println("Il valore della variabile è " + n);
3  System.out.println("n = " + n + ", doppio = " + (n * 2));
4
5  System.out.println("n + 1 = " + n + 1);   // Concatenazione: "42" poi "1"
6  System.out.println("n + 1 = " + (n + 1)); // Addizione, poi
       concatenazione
7
8  System.out.print("Niente a capo ");
9  System.out.println("dopo print");
10 System.out.printf("%d diviso %d è %.2f%n", 10, 3, 10.0 / 3);
```

Esempio (2/2)

Output:

```
Il valore della variabile è 42
n = 42, doppio = 84
n + 1 = 421
n + 1 = 43
Niente a capo dopo print
10 diviso 3 è 3.33
```

Il separatore decimale

- Il separatore decimale stampato da `printf` dipende dalle impostazioni della lingua del calcolatore
- Su un calcolatore impostato in italiano l'ultima riga dell'esempio è

  ```
  10 diviso 3 è 3,33
  ```

  con la virgola
- Il metodo `println` usa sempre il punto

Ingresso da tastiera

- Per leggere l'ingresso da tastiera si usa la classe `Scanner` del pacchetto `java.util`
- Va importata con l'istruzione

```
import java.util.Scanner;
```

  scritta all'inizio del file, prima della classe
- I metodi `nextLine()`, `nextInt()` e `nextDouble()` leggono una riga di testo, un intero e un numero in virgola mobile

Esempio (1/3)

Ingresso da tastiera con `Scanner`

```
1 import java.util.Scanner;
2
3 public class InputExample {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.print(" Inserisci il tuo nome: ");
9         String name = sc.nextLine();       // Legge una String
```

Esempio (2/3)

```
11          System.out.print(" Inserisci un intero: ");
12          int n = sc.nextInt();                  // Legge un int
13
14          System.out.print(" Inserisci un numero reale: ");
15          double x = sc.nextDouble();            // Legge un double
16
17          System.out.println("Ciao " + name + ", n = " + n + ", x = " + x
        );
18          sc.close();
19      }
20  }
```

Esempio (3/3)

Esempio di esecuzione (l'utente scrive `Ada`, `7` e `2.5`):

```
Inserisci il tuo nome: Ada
Inserisci un intero: 7
Inserisci un numero reale: 2.5
Ciao Ada, n = 7, x = 2.5
```

- Come `printf`, il metodo `nextDouble()` segue le impostazioni della lingua: su un calcolatore impostato in italiano il numero reale va scritto come `2,5`

Riepilogo

Uscita:

- `System.out.println()` e `System.out.print()`, con o senza l'a capo
- concatenazione con +, valutata da sinistra a destra: l'aritmetica fra parentesi
- `System.out.printf()` con `%d`, `%s`, `%.kf`, `%n`; il suo separatore decimale segue le impostazioni della lingua

Ingresso:

- la classe `Scanner` di `java.util`, con `nextLine()`, `nextInt()` e `nextDouble()`

Operatori

Introduzione: operatori

- Gli operatori sono simboli che eseguono operazioni su variabili e valori
- Gli operatori di `Java` sono gli stessi di `C++`:
  - operatori aritmetici
  - operatori di assegnamento
  - operatori di incremento e decremento
  - operatori relazionali
  - operatori logici
- Per ogni famiglia vediamo i simboli e un esempio con il suo output

Operatori aritmetici

Operatori aritmetici, usati per eseguire le operazioni matematiche di base:

- + (Addizione) $\rightsquigarrow$ Somma due valori
- - (Sottrazione) $\rightsquigarrow$ Sottrae un valore da un altro
- * (Moltiplicazione) $\rightsquigarrow$ Moltiplica due valori
- / (Divisione) $\rightsquigarrow$ Divide un valore per un altro. Divisione intera quando entrambi gli operandi sono interi
- % (Resto) $\rightsquigarrow$ Restituisce il resto di una divisione

Esempio (1/2)

Operatori aritmetici

```
1 int a = 10, b = 3;
2
3 System.out.println("Addizione:          " + (a + b));
4 System.out.println("Sottrazione:        " + (a - b));
5 System.out.println("Moltiplicazione:    " + (a * b));
6 System.out.println("Divisione:          " + (a / b));   // Divisione intera
7 System.out.println("Resto:              " + (a % b));   // Resto della
      divisione
8 System.out.println("Divisione reale:    " + ((double) a / b));
```

Esempio (2/2)

Output:

```
Addizione:       13
Sottrazione:     7
Moltiplicazione: 30
Divisione:       3
Resto:           1
Divisione reale: 3.3333333333333335
```

Divisione intera e conversione di tipo

- Come in C++, l'operatore di divisione / esegue la **divisione intera** quando entrambi gli operandi sono interi, e il quoziente è troncato verso lo zero:
  - `10 / 3` vale 3
  - `-7 / 2` vale `-3` (con resto `-7 % 2` uguale a `-1`)

- Per ottenere un risultato in virgola mobile almeno un operando deve essere di tipo `double`, per esempio con la **conversione di tipo (cast)** `(double) a / b`

- La conversione `(int)` elimina la parte decimale di un `double`: `(int) 3.9` vale 3

- Non esiste un operatore per la potenza: il metodo `Math.pow(x, y)` restituisce x^y come `double`

Esempio (1/2)

Divisione intera e in virgola mobile

```
1 int a = 10, b = 3;
2
3 System.out.println(a / b);                    // Divisione intera
4 System.out.println(10.0 / 3);                 // Divisione in virgola mobile
5 System.out.println((double) a / b);           // Conversione di a in double
6 System.out.printf("%.2f%n", (double) a / b);  // Due cifre decimali
7 System.out.println(-7 / 2 + " " + -7 % 2);    // Troncamento verso zero
8 System.out.println((int) 3.9 + " " + Math.pow(2, 10));
```

Esempio (2/2)

Output:

```
3
3.3333333333333335
3.3333333333333335
3.33
-3 -1
3 1024.0
```

Operatori di assegnamento

Operatori di assegnamento, usati per assegnare valori alle variabili:

- = (Assegnamento) $\rightsquigarrow$ Assegna un valore a una variabile
- += (Assegnamento con addizione) $\rightsquigarrow$ Somma un valore e assegna il risultato
- -= (Assegnamento con sottrazione) $\rightsquigarrow$ Sottrae un valore e assegna il risultato
- *= (Assegnamento con moltiplicazione) $\rightsquigarrow$ Moltiplica e assegna il risultato
- /= (Assegnamento con divisione) $\rightsquigarrow$ Divide e assegna il risultato
- %= (Assegnamento con resto) $\rightsquigarrow$ Calcola il resto e assegna il risultato

Esempio (1/2)

Operatori di assegnamento

```
1 int x = 10; // Assegnamento
2 System.out.println("Valore iniziale : " + x);
3
4 x += 5; System.out.println("Dopo += 5: " + x); // x = x + 5
5 x -= 3; System.out.println("Dopo -= 3: " + x); // x = x - 3
6 x *= 2; System.out.println("Dopo *= 2: " + x); // x = x * 2
7 x /= 4; System.out.println("Dopo /= 4: " + x); // x = x / 4
8 x %= 4; System.out.println("Dopo %= 4: " + x); // x = x % 4
```

Esempio (2/2)

Output:

```
Valore iniziale: 10
Dopo += 5: 15
Dopo -= 3: 12
Dopo *= 2: 24
Dopo /= 4: 6
Dopo %= 4: 2
```

- Due istruzioni possono essere scritte sulla stessa riga, perché ciascuna termina con ";"

Operatori di incremento e decremento

Operatori di incremento e decremento:

- ++ (Incremento) $\rightsquigarrow$ Aumenta di 1 il valore di una variabile
- - - (Decremento) $\rightsquigarrow$ Diminuisce di 1 il valore di una variabile

- Come in `C++`, il **pre-incremento** (`++x`) incrementa il valore prima di restituirlo
- Il **post-incremento** (`x++`) restituisce prima il valore, poi lo incrementa
- La stessa logica vale per l'operatore di decremento (`- -x` e `x- -`)

Esempio (1/2)

Operatori di incremento e decremento

```
1 int x = 5;
2 System.out.println("Valore iniziale : " + x);
3 System.out.println("Pre-incremento (++x): " + ++x);
4 System.out.println("Post-incremento (x++): " + x++);
5 System.out.println("Valore dopo il post-incremento: " + x);
6 System.out.println("Pre-decremento (--x): " + --x);
7 System.out.println("Post-decremento (x--): " + x--);
8 System.out.println("Valore dopo il post-decremento: " + x);
```

Esempio (2/2)

Output:

```
Valore iniziale: 5
Pre-incremento (++x): 6
Post-incremento (x++): 6
Valore dopo il post-incremento: 7
Pre-decremento (--x): 6
Post-decremento (x--): 6
Valore dopo il post-decremento: 5
```

Operatori relazionali

Operatori relazionali, usati per confrontare valori e restituire un risultato `boolean`:

- == (Uguale a) $\rightsquigarrow$ Verifica se due valori sono uguali
- != (Diverso da) $\rightsquigarrow$ Verifica se due valori sono diversi
- < (Minore di) $\rightsquigarrow$ Verifica se un valore è minore di un altro
- > (Maggiore di) $\rightsquigarrow$ Verifica se un valore è maggiore di un altro
- <= (Minore o uguale a) $\rightsquigarrow$ Verifica se un valore è minore o uguale a un altro
- >= (Maggiore o uguale a) $\rightsquigarrow$ Verifica se un valore è maggiore o uguale a un altro

Risultati booleani e stringhe

- Gli operatori relazionali restituiscono un valore di tipo `boolean`, visualizzato come `true` o `false` (non come `1` e `0` come in `C++`)
- Questi operatori sono usati comunemente nelle **istruzioni condizionali** e nei **cicli**
- L'operatore == confronta i valori dei tipi di dato primitivi (`int`, `double`, `char`, `boolean`)
- Due valori `String` si confrontano con il metodo `equals`, per esempio

```
name.equals("Alice")
```

Esempio (1/2)

Operatori relazionali

```
1 int a = 10, b = 5;
2
3 System.out.println("a == b: " + (a == b)); // Uguale a
4 System.out.println("a != b: " + (a != b)); // Diverso da
5 System.out.println("a < b: " + (a < b));   // Minore di
6 System.out.println("a > b: " + (a > b));   // Maggiore di
7 System.out.println("a <= b: " + (a <= b)); // Minore o uguale a
8 System.out.println("a >= b: " + (a >= b)); // Maggiore o uguale a
```

Esempio (2/2)

Output:

```
a == b: false
a != b: true
a < b: false
a > b: true
a <= b: false
a >= b: true
```

Operatori logici

Operatori logici, usati per le operazioni logiche, spesso nelle istruzioni condizionali:

- `&&` (AND logico) $\rightsquigarrow$ Restituisce `true` se entrambe le condizioni sono `true`
- `||` (OR logico) $\rightsquigarrow$ Restituisce `true` se almeno una condizione è `true`
- `!` (NOT logico) $\rightsquigarrow$ Inverte il valore booleano

Esempio (1/2)

Operatori logici

```
1 boolean c1 = true, c2 = false;
2
3 System.out.println("c1: " + c1);
4 System.out.println("c2: " + c2);
5 System.out.println("c1 && c2: " + (c1 && c2)); // true solo se entrambe
      sono true
6 System.out.println("c1 || c2: " + (c1 || c2)); // true se almeno una è
      true
7 System.out.println("!c1: " + (!c1));           // Inverte il valore
8 System.out.println("!c2: " + (!c2));
```

Esempio (2/2)

Output:

```
c1: true
c2: false
c1 && c2: false
c1 || c2: true
!c1: false
!c2: true
```

Riepilogo (1/2)

Operatori aritmetici:

- `+`, `-`, `*`, `/`, `%`; fra due interi `/` è la divisione intera, troncata verso lo zero
- il cast `(double)` dà un risultato in virgola mobile, `(int)` elimina la parte decimale; `Math.pow(x, y)` per la potenza

Assegnamento, incremento e decremento:

- `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- `++x` incrementa prima di restituire il valore, `x++` dopo

Riepilogo (2/2)

Operatori relazionali:

- `==`, `!=`, `<`, `>`, `<=`, `>=` restituiscono un `boolean`, stampato come `true` o `false`
- due stringhe si confrontano con `equals`

Operatori logici:

- `&&`, `||`, `!`, come in C++

Istruzioni condizionali

Introduzione: istruzioni condizionali

- Gli operatori relazionali e logici producono le **condizioni** che decidono quali istruzioni vengono eseguite
- In questa sezione vediamo le istruzioni `if`, `else if` ed `else`
- Poi vediamo l'operatore ternario, che sceglie fra due valori in una sola espressione

L'istruzione `if`

- L'istruzione `if` valuta una condizione racchiusa fra parentesi `()`
- Se la condizione è `true`, viene eseguito il blocco racchiuso fra parentesi graffe `{}`
- La clausola opzionale `else if` verifica condizioni aggiuntive, e la clausola opzionale `else` tratta tutti i casi rimanenti
- A differenza di C++, la condizione deve essere di tipo `boolean`: un intero non è accettato come condizione
  - `if (n % 2)` è un errore di compilazione
  - il test va scritto `if (n % 2 != 0)`

Esempio

Istruzione condizionale

```
1 int number = 10;
2
3 // Verifica se il numero è positivo
4 if (number > 0) {
5     System.out.println("Il numero è positivo.");
6 } else {
7     System.out.println("Il numero non è positivo.");
8 }
```

Output:

```
Il numero è positivo.
```

Esempio (1/2)

`if / else if / else`

```
1  int score = 85;
2  String grade;
3
4  if      (score >= 90) { grade = "A"; }
5  else if (score >= 80) { grade = "B"; }
6  else if (score >= 70) { grade = "C"; }
7  else if (score >= 60) { grade = "D"; }
8  else                  { grade = "F"; }
9
10 System.out.println ("Punteggio: " + score + ", Voto: " + grade);
```

Output:

```
Punteggio: 85, Voto: B
```

Esempio (2/2)

- Le condizioni sono valutate dall'alto verso il basso, e viene eseguito solo il blocco della prima condizione vera
- La variabile `grade` è dichiarata senza un valore: il compilatore la accetta perché ogni ramo le assegna un valore prima che venga stampata

Operatore ternario (operatore condizionale)

- L'**operatore ternario** `?:` è lo stesso di C++
- Sintassi:

```
condizione ? valore_se_vero : valore_se_falso
```

- L'espressione prende il primo valore quando la condizione è `true`, il secondo altrimenti

Esempio (1/3)

Operatore ternario

```
1  int number = 10;
2
3  // condizione ? valore_se_vero : valore_se_falso
4  String result = (number > 0) ? "positivo" : "non positivo";
5  System.out.println("Il numero è " + result);
6
7  // Ricerca del massimo fra due numeri
8  int a = 15, b = 20;
9  int maximum = (a > b) ? a : b;
10 System.out.println("Massimo: " + maximum);
```

Esempio (2/3)

```
12 // Verifica se pari o dispari
13 int n = 7;
14 String parity = (n % 2 == 0) ? "pari" : "dispari";
15 System.out.println(n + " è " + parity);
16
17 // Valore assoluto
18 int x = -5;
19 int absValue = (x >= 0) ? x : -x;
20 System.out.println("Il valore assoluto di " + x + " è " + absValue);
```

Esempio (3/3)

Output:

```
Il numero è positivo
Massimo: 20
7 è dispari
Il valore assoluto di -5 è 5
```

- La classe `Math` fornisce anche i metodi `Math.max(a, b)` e `Math.abs(x)`, che restituiscono gli stessi valori

Riepilogo

Istruzioni condizionali:

- `if`, `else if`, `else`: viene eseguito solo il blocco della prima condizione vera
- la condizione deve essere di tipo `boolean`: `if (n % 2 != 0)`, non `if (n % 2)`

Operatore ternario:

- `condizione ? valore_se_vero : valore_se_falso`, come in C++
- `Math.max` e `Math.abs` danno il massimo e il valore assoluto

Cicli

Introduzione: cicli

- Le istruzioni condizionali scelgono *se* un blocco di codice viene eseguito; i cicli decidono *quante volte* viene eseguito
- I cicli servono a eseguire ripetutamente un blocco di codice finché una condizione è soddisfatta
- Come in `C++`, i cicli più usati di `Java` sono il ciclo `while` e il ciclo `for`; vediamo anche `break` e `continue`

Il ciclo `while`

- Il ciclo `while` esegue un blocco di codice finché la condizione racchiusa fra parentesi `()` vale `true`. Quando la condizione è `false`, il ciclo termina

Esempio

```
1 int i = 1; // Inizializzazione
2
3 while (i <= 5) { // Condizione
4     System.out.println("Numero: " + i);
5     i++; // Incremento
6 }
```

Output:

```
Numero: 1
Numero: 2
Numero: 3
Numero: 4
Numero: 5
```

Il ciclo `for`

- Il ciclo `for` combina inizializzazione, condizione e incremento (o decremento) in una sola istruzione, esattamente come in C++

Esempio

```
1 //  Inizializzazione  ;  Condizione;  Incremento
2 for  (int  i  = 1; i  <= 5; i++) {
3     System.out. println ("Numero: " + i);
4 }
```

L'output è lo stesso del ciclo `while` dell'esempio precedente.

Le istruzioni `break` e `continue`

- L'istruzione `break` termina immediatamente il ciclo
- L'istruzione `continue` salta il resto dell'iterazione corrente e passa alla successiva

Esempio

`break` **e** `continue`

```java
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // Salta i numeri pari
    }
    if (i > 7) {
        break; // Interrompe il ciclo al primo dispari maggiore di 7
    }
    System.out.print(i + " ");
}
System.out.println();
```

Output:

```
1 3 5 7
```

L'output è lo stesso delle versioni `C++` e `Python`: il valore 9 fa scattare il `break`.

Riepilogo

Cicli:

- `while`: ripete un blocco finché la condizione vale `true`
- `for`: inizializzazione, condizione e incremento in una sola istruzione, come in C++

Salti:

- `break` termina il ciclo
- `continue` passa all'iterazione successiva

Array

Introduzione: array

- I cicli diventano davvero utili quando visitano **collezioni** di valori
- Una collezione di variabili dello stesso tipo di dato è memorizzata in un **array**
- In questa sezione vediamo gli array a una e a due dimensioni, e le liste di dimensione variabile della classe `ArrayList`
- Gli array sono l'input di quasi tutti gli algoritmi del corso

Gli array in Java (1/2)

- Un **array** in Java è una collezione ordinata di elementi dello stesso tipo di dato, con una dimensione fissata al momento della creazione: la dimensione non può cambiare in seguito
- Il tipo di dato di un array di interi si scrive `int[]`
- Un array si alloca con l'operatore `new`, per esempio `new int[n]`, dove la dimensione `n` può essere decisa durante l'esecuzione, come per gli array di dimensione dinamica di C++
- Gli elementi di un nuovo array sono inizializzati a `0` (a `false` per `boolean`)

Gli array in Java (2/2)

- Gli elementi si raggiungono tramite il loro **indice** fra parentesi quadre `[]`, a partire da `0`: gli elementi di un array di dimensione n sono

  `a[0]`, `a[1]`, ..., `a[n-1]`

- Ogni array conosce la propria dimensione: la lunghezza di un array `a` è `a.length`

- A differenza di `C++`, la memoria di un array non deve essere liberata con `delete[]`: viene rilasciata automaticamente dal **garbage collector** della JVM quando l'array non è più usato

Array: l'essenziale

Gli array si allocano con `new` prima dell'uso
(oppure si creano con un elenco di valori fra parentesi graffe).

Gli indici partono da `0`,
la dimensione è fissa ed è data da `a.length`.

Esempio (1/2)

Creazione e accesso a un array

```
1  int n = 5;
2
3  int [] a = new int[n]; // Allocazione di un array di n interi
4
5  for (int i = 0; i < n; i++) {
6      a[i] = i * 2; // Esempio di assegnamento di valori
7  }
8
9  System.out.println("Lunghezza: " + a.length);
10 System.out.println("a[0] = " + a[0]); // Primo elemento
11 System.out.println("a[4] = " + a[4]); // Ultimo elemento, indice n-1
```

Output:

```
Lunghezza: 5
a[0] = 0
a[4] = 8
```

Esempio (2/2)

- Accedere a un indice che non esiste, per esempio `a[5]`, ferma il programma con l'errore

```
ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 5
```

- In C++ lo stesso accesso non è controllato

Array scritti per esteso e ciclo `for` esteso

- Un array i cui valori sono noti quando si scrive il programma può essere creato elencando i suoi valori fra parentesi graffe, come in C++
- Il **ciclo `for` esteso** `for (int x : a)` visita direttamente gli elementi dell'array `a`, senza usare i loro indici, come il ciclo `for` su una lista di Python
- Il metodo `Arrays.toString(a)`, della classe `Arrays` del pacchetto `java.util`, restituisce gli elementi di un array come un testo fra parentesi quadre

Esempio (1/2)

Array scritto per esteso e iterazione

```
1  import java.util.Arrays;
2
3  int[] a = {9, 12, 3, 16, 19, 11, 12};
4  int n = a.length;
5
6  for (int i = 0; i < n; i++) {      // Ciclo sugli indici
7      System.out.println("a[" + i + "] = " + a[i]);
8  }
9
10 for (int x : a) {                  // Ciclo sugli elementi
11     System.out.print(x + " ");
12 }
13 System.out.println();
14
15 System.out.println(Arrays.toString(a));
```

Esempio (2/2)

Output:

```
a[0] = 9
a[1] = 12
a[2] = 3
a[3] = 16
a[4] = 19
a[5] = 11
a[6] = 12
9 12 3 16 19 11 12
[9, 12, 3, 16, 19, 11, 12]
```

L'istruzione `import` si scrive all'inizio del file, prima della classe.

Array bidimensionali (1/2)

- Gli array bidimensionali (matrici) si dichiarano con due coppie di parentesi quadre `[][]`
- Si allocano con `new int[m][n]` (*m* righe e *n* colonne) in una sola istruzione
- L'elemento nella riga *i* e colonna *j* si raggiunge come `matrix[i][j]`, con entrambi gli indici a partire da 0
- `matrix.length` è il numero di righe e `matrix[i].length` è il numero di colonne della riga *i*

Array bidimensionali (2/2)

- Un array bidimensionale può anche essere creato elencando i suoi valori riga per riga:

```
int[][] c = {{3, 1, 4}, {1, 5, 9}};
```

  crea un array con 2 righe e 3 colonne

- `c[1][2]` è l'elemento nella seconda riga e terza colonna, uguale a 9

Esempio (1/2)

Array bidimensionale (matrice)

```
1  int m = 3; // Numero di righe
2  int n = 2; // Numero di colonne
3
4  int [][] matrix = new int[m][n]; // Allocazione di una matrice m x n
5
6  for (int i = 0; i < m; i++) {
7      for (int j = 0; j < n; j++) {
8          matrix[i][j] = i * j; // Esempio di assegnamento di valori
9      }
10 }
11
12 for (int i = 0; i < m; i++) {
13     for (int j = 0; j < n; j++) {
14         System.out.println("matrix[" + i + "][" + j + "] = " + matrix[i][
       j]);
15     }
16 }
```

Esempio (2/2)

Output:

```
matrix[0][0] = 0
matrix[0][1] = 0
matrix[1][0] = 0
matrix[1][1] = 1
matrix[2][0] = 0
matrix[2][1] = 2
```

Liste di dimensione variabile: `ArrayList`

- Quando il numero di elementi non è noto in anticipo, la classe `ArrayList` del pacchetto `java.util` fornisce una lista che cresce durante l'esecuzione, come le liste di `Python`
- Il tipo di dato di una lista di interi è `ArrayList<Integer>`: gli elementi di una `ArrayList` sono oggetti, e `Integer` è la classe che corrisponde a `int`
- Il metodo `add(x)` aggiunge l'elemento `x` in fondo alla lista, `get(i)` restituisce l'elemento di indice i, `set(i, x)` lo sostituisce, e `size()` restituisce il numero di elementi

Esempio

Una lista di dimensione variabile

```
1  import java.util.ArrayList;
2
3  ArrayList<Integer> list = new ArrayList<>(); // Lista vuota
4
5  for (int i = 1; i <= 4; i++) {
6      list.add(i * i);   // La lista cresce
7  }
8  list.set(0, 100);      // Sostituisce l'elemento di indice 0
9
10 System.out.println( list + " ha " + list.size() + " elementi");
11 System.out.println(" list.get(2) = " + list.get(2));
```

Output:

```
[100, 4, 9, 16] ha 4 elementi
list.get(2) = 9
```

Riepilogo

Array:

- `int[] a = new int[n];` oppure un elenco di valori fra parentesi graffe; la dimensione è fissa
- indici da `0` a `a.length - 1`; un indice fuori intervallo ferma il programma con un'eccezione
- il ciclo `for` esteso e `Arrays.toString(a)`; memoria liberata dal garbage collector

Matrici e liste:

- `new int[m][n]`, elemento `matrix[i][j]`
- `ArrayList<Integer>` cresce con `add`; `get`, `set`, `size`

Metodi (funzioni)

Introduzione: metodi (funzioni)

- Come nello pseudocodice, un algoritmo è scritto come una funzione che riceve l'input e restituisce l'output
- In `Java` le funzioni si chiamano **metodi** e devono sempre essere definite dentro una **classe**
- In questa sezione vediamo come si scrive e si chiama un metodo, e come funzionano i suoi parametri e i suoi valori di ritorno

Le parti di un metodo

Un metodo in `Java` è composto da:

1. un **modificatore di accesso** opzionale (ad esempio `public`, `private`) e la parola chiave `static` quando il metodo appartiene alla classe invece che a un oggetto della classe (tutti i metodi di questo capitolo sono `static`)
2. un **tipo di ritorno**: specifica il tipo del valore restituito (`void` se non viene restituito nulla)
3. un **nome del metodo**: l'identificatore univoco del metodo
4. un **elenco di parametri** (opzionale): specifica i valori in ingresso, ciascuno con il suo tipo di dato
5. un **corpo**: il blocco di codice eseguito quando il metodo viene chiamato
6. un'istruzione `return`, obbligatoria se il tipo di ritorno non è `void`

Il punto di ingresso e l'ordine dei metodi

- Il **punto di ingresso** di ogni programma `Java` è il metodo

  ```
  public static void main(String[] args)
  ```

  che svolge il ruolo della funzione `main` di `C++`
- Il parametro `args` contiene le parole scritte dopo il nome della classe nel comando `java`

- A differenza di `C++`, un metodo può essere chiamato prima del punto in cui è definito nella classe: l'ordine dei metodi dentro una classe non conta

- Il codice sorgente `Java` può anche essere compilato ed eseguito online, per esempio all'indirizzo `https://www.mycompiler.io/online-java-compiler`

Esempio (1/2)

Metodi

```
1  public class Main {
2
3      // Definizione di metodo
4      static int addition(int a, int b) {
5          return a + b; // Restituisce la somma
6      }
7
8      // Definizione di metodo
9      static int multiplication (int a, int b) {
10         return a * b; // Restituisce il prodotto
11     }
```

Esempio (2/2)

```
13      public static void main(String[] args) {
14          int sum = addition(5, 10);              // Chiamata di metodo
15          System.out.println("Somma: " + sum);
16
17          int prod = multiplication(5, 10);   // Chiamata di metodo
18          System.out.println("Prodotto: " + prod);
19      }
20  }
```

Output:

```
Somma: 15
Prodotto: 50
```

Esempio (1/2)

Metodo che opera su un array bidimensionale

```
1  public class Main {
2
3      static int sumMatrix(int [][] matrix, int rows, int cols) {
4          int sum = 0;
5          for (int i = 0; i < rows; i++) {
6              for (int j = 0; j < cols; j++) {
7                  sum += matrix[i][j];
8              }
9          }
10         return sum;
11     }
```

Esempio (2/2)

```
13      public static void main(String[] args) {
14          int m = 3, n = 2;
15          int [][] matrix = new int[m][n];
16
17          for (int i = 0; i < m; i++)
18              for (int j = 0; j < n; j++)
19                  matrix[i][j] = i * j;
20
21          int total = sumMatrix(matrix, m, n);
22          System.out.println("Somma degli elementi della matrice: " + total
        );
23      }
24  }
```

Output:

```
Somma degli elementi della matrice: 3
```

Il corpo di un ciclo `for` fatto di una sola istruzione può essere scritto senza parentesi graffe, come in C++.

Parametri e valori di ritorno (1/2)

- In `Java` ogni parametro è passato **per valore**: il metodo riceve una copia dell'argomento
- Per i tipi di dato primitivi (`int`, `double`, `boolean`, `char`) le modifiche fatte alla copia non influiscono sulla variabile di chi chiama
- Non esiste il passaggio per riferimento con il simbolo `&` di `C++`

Parametri e valori di ritorno (2/2)

- Una variabile di tipo `int[]` non contiene gli elementi, ma un **riferimento** all'array (simile al puntatore di C++)
- Il metodo riceve una copia del riferimento, che punta allo stesso array: le modifiche fatte agli *elementi* dell'array sono visibili a chi chiama
- Un metodo restituisce al più un valore. Quando un metodo deve restituire più risultati, può restituirli in un array, per esempio `new int[]{q, r}`

Esempio (1/3)

Parametri e risultati multipli

```
1  public class Main {
2
3      static void increment(int x) {
4          x = x + 1; // Modifica solo la copia locale
5      }
6
7      static void fillWithZeros(int[] a) {
8          for (int i = 0; i < a.length; i++) {
9              a[i] = 0; // Modifica l'array di chi chiama
10         }
11     }
12
13     static int[] divide(int n, int m) {
14         return new int[]{n / m, n % m}; // Due risultati in un array
15     }
```

Esempio (2/3)

```
17      public static void main(String[] args) {
18          int k = 5;
19          increment(k);
20          System.out.println("Dopo increment: " + k);
21
22          int[] b = {4, 7, 1};
23          fillWithZeros(b);
24          System.out.println("Array: " + b[0] + " " + b[1] + " " + b[2]);
25
26          int[] qr = divide(17, 5);
27          System.out.println("Quoziente e resto: " + qr[0] + " " + qr[1]);
28      }
29  }
```

Esempio (3/3)

Output:

```
Dopo increment: 5
Array: 0 0 0
Quoziente e resto: 3 2
```

- `increment` modifica solo la sua copia: `k` vale ancora 5
- `fillWithZeros` riceve una copia del riferimento a `b`, e mette a 0 gli elementi dello stesso array
- `divide` restituisce in un array quoziente e resto della divisione di 17 per 5

Riepilogo

Metodi:

- definiti dentro una classe: modificatori, tipo di ritorno, nome, parametri, corpo, `return`
- il punto di ingresso è `public static void main(String[] args)`; l'ordine dei metodi non conta

Parametri e valori di ritorno:

- ogni parametro è passato per valore; un array è passato tramite una copia del suo riferimento, quindi i suoi elementi possono essere modificati
- più risultati si restituiscono insieme in un array

Un algoritmo completo: il valore massimo in un array

Introduzione: un algoritmo completo

- Mettiamo ora insieme tutti i pezzi: un metodo, un ciclo, un'istruzione condizionale e un array
- Traduciamo in `Java` lo pseudocodice di un algoritmo, facendo attenzione agli indici
- Poi lo proviamo su un'istanza fissata e su una casuale, e seguiamo la sua esecuzione passo per passo

Il problema

Consideriamo il problema di trovare il valore massimo in un array a di $n \geq 1$ numeri interi.

- La slide seguente mostra lo pseudocodice dell'algoritmo `FindMaximum` del capitolo «Pseudocodice»: le componenti sono indicizzate da 1 a n

Lo pseudocodice

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: Il valore massimo nell'array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

Dallo pseudocodice a Java

- Nell'implementazione in `Java` gli elementi sono indicizzati da 0 a $n-1$, quindi l'inizializzazione usa `a[0]` e il ciclo parte da `i = 1`
- La dimensione n non è passata come parametro, perché il metodo la ottiene con `a.length`
- Il metodo suppone $n \geq 1$, come lo pseudocodice: per un array vuoto l'accesso ad `a[0]` ferma il programma con una `ArrayIndexOutOfBoundsException`

L'implementazione in Java (1/2)

```java
public class Main {

    // Metodo che trova il valore massimo in un array
    static int arrayMaxValue(int[] a) {
        int max = a[0]; // Inizializza con il primo elemento

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) {
                max = a[i]; // Aggiorna il massimo
            }
        }

        return max;
    }
```

L'implementazione in Java (2/2)

```
16      public static void main(String[] args) {
17          int[] a = {9, 12, 3, 16, 19, 11, 12};
18
19          int max = arrayMaxValue(a);
20          System.out.println("Valore massimo: " + max);
21      }
22  }
```

Output:

```
Valore massimo: 19
```

Un'istanza casuale

- L'istanza di prova può anche essere generata a caso con la classe `Random` del pacchetto `java.util`
- Se `rand` è un oggetto di questa classe, `rand.nextInt(100)` restituisce un intero casuale fra 0 e 99, quindi `rand.nextInt(100) + 1` è fra 1 e 100
- L'oggetto creato con `new Random(2026)` parte dal valore fisso 2026 (il **seme**), in modo che ogni esecuzione produca lo stesso array
- L'oggetto creato con `new Random()` produce un array diverso a ogni esecuzione

Esempio (1/2)

Un'istanza casuale

```
1  import java.util.Arrays;
2  import java.util.Random;
3
4  public class Main {
5
6      static int arrayMaxValue(int[] a) {
7          int max = a[0];
8          for (int i = 1; i < a.length; i++) {
9              if (a[i] > max) {
10                 max = a[i];
11             }
12         }
13         return max;
14     }
```

Esempio (2/2)

```
16      public static void main(String[] args) {
17          int n = 10; // Dimensione dell'array
18          int[] a = new int[n];
19          Random rand = new Random(2026); // Stesso array casuale a ogni
        esecuzione
20          for (int i = 0; i < n; i++) {
21              a[i] = rand.nextInt(100) + 1; // Intero casuale fra 1 e 100
22          }
23
24          System.out.println(Arrays.toString(a));
25          System.out.println("Valore massimo: " + arrayMaxValue(a));
26      }
27  }
```

Output:

```
[100, 51, 98, 31, 50, 39, 74, 37, 35, 79]
Valore massimo: 100
```

Esempio (1/3)

Esecuzione passo per passo

Seguiamo l'esecuzione di `arrayMaxValue` su due istanze, con una tabella per ciascuna:

- la colonna i è l'indice del ciclo `Java`
- la colonna $i + 1$ è l'indice corrispondente dello pseudocodice
- la prima riga è l'inizializzazione `max = a[0]`

Esempio (2/3)

Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (Java)	$i + 1$ (pseudocodice)	`a[i]`	`a[i] > max`?	`max`
–	–	–	–	9
1	2	12	Sì	12
2	3	3	No	12
3	4	16	Sì	16
4	5	19	Sì	19
5	6	11	No	19
6	7	12	No	19

Restituisce: `max` $= 19$

Esempio (3/3)

Istanza 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (Java)	$i + 1$ (pseudocodice)	`a[i]`	`a[i] > max`?	`max`
–	–	–	–	3
1	2	7	Sì	7
2	3	2	No	7
3	4	9	Sì	9
4	5	1	No	9

Restituisce: `max` $= 9$

Riepilogo

Dallo pseudocodice a `Java`:

- $a[1]$ diventa `a[0]`, il ciclo da 2 a n diventa il ciclo da `1` a `a.length - 1`
- la dimensione si ottiene con `a.length`; il metodo suppone $n \geq 1$

Prove:

- su un array fissato, e su uno casuale generato con `Random`: con un seme ogni esecuzione produce lo stesso array
- le tabelle passo per passo seguono l'indice i di `Java` e $i + 1$ dello pseudocodice

Operatori bit a bit

Introduzione: operatori bit a bit

- Gli operatori bit a bit lavorano sulla rappresentazione binaria degli interi
- Esistono in `Java` come in `C++`: in questa sezione vediamo le poche differenze, che dipendono dai 32 bit di `int` e dalle condizioni di tipo `boolean`

Gli stessi operatori di C++

- Gli operatori bit a bit del capitolo «Introduzione a C++» esistono in Java con gli stessi simboli e lo stesso significato:
  - `&` (AND), `|` (OR), `^` (XOR), `~` (NOT)
  - `<<` (scorrimento a sinistra) e `>>` (scorrimento a destra)

- Il metodo `Integer.toBinaryString(x)` restituisce la rappresentazione binaria di un intero

Che cosa cambia in Java

- Un valore di tipo `int` ha 32 bit, quindi `~12` inverte 32 bit e dà `-13`; il risultato a 8 bit di `C++` si ottiene con la maschera `0xFF`, come in `Python`
- `Java` ha un terzo operatore di scorrimento, `>>>` (scorrimento a destra senza segno), che aggiunge sempre zeri a sinistra, anche per i numeri negativi; per i numeri non negativi coincide con `>>`
- Poiché una condizione deve essere di tipo `boolean`, il test del bit meno significativo si scrive `(n & 1) == 1`, e non `(n & 1)` come in `C++`

Esempio (1/2)

Operatori bit a bit

```
1  int a = 12; // Binario: 1100
2  int b = 10; // Binario: 1010
3
4  System.out.println((a & b) + " " + (a | b) + " " + (a ^ b)); // AND, OR,
       XOR
5  System.out.println(Integer.toBinaryString(a & b));              // 1000
6  System.out.println((5 << 2) + " " + (20 >> 2) + " " + (20 >> 3)); //
       Scorrimenti
7  System.out.println(~a + " " + (~a & 0xFF));  // NOT, e la sua versione a
       8 bit
8  System.out.println((-8 >> 1) + " " + (-8 >>> 28)); // Scorrimento con
       e senza segno
9
10 int n = 15;
11 System.out.println(((n & 1) == 1) ? "dispari" : "pari"); // Bit meno
       significativo
```

Esempio (2/2)

Output:

```
8 14 6
1000
20 5 2
-13 243
-4 15
dispari
```

- Il numero -8 è memorizzato come $11\ldots11000$ (32 bit)
- `-8 >> 1` mantiene il segno e dà -4
- `-8 >>> 28` mantiene solo i 4 bit più a sinistra 1111, cioè 15

Riepilogo

Operatori bit a bit:

- `&`, `|`, `^`, `~`, `<<`, `>>` come in C++; `Integer.toBinaryString(x)` mostra i bit
- un `int` ha 32 bit: `~12` vale `-13`, e la maschera `0xFF` dà il risultato a 8 bit

Solo in `Java`:

- lo scorrimento a destra senza segno `>>>`, che aggiunge zeri a sinistra
- il test del bit meno significativo è `(n & 1) == 1`

Java, C++ e Python a confronto

Introduzione: Java, C++ e Python a confronto

- Abbiamo visto i tre linguaggi di questa parte del corso
- La tabella seguente riassume le principali differenze fra di loro, riga per riga
- È un riferimento utile quando un algoritmo scritto in pseudocodice va tradotto in uno dei tre linguaggi

Tabella di confronto (1/3)

	C++	Java	Python
Esecuzione	compilato (g++)	compilato in bytecode (`javac`), eseguito dalla JVM (`java`)	interpretato (`python3`)
Tipi di dato	dichiarati	dichiarati	non dichiarati
Blocchi	`{}`	`{}`	`:` e indentazione
Fine dell'istruzione	;	;	fine della riga

Tabella di confronto (2/3)

	C++	Java	Python
Valori booleani	`true`, `false` (stampati `1`, `0`)	`true`, `false`	`True`, `False`
`10 / 3`	`3`	`3`	`3.3333333333333335`
`-7 / 2` (intero)	`-3`	`-3`	`-7 // 2` vale `-4`
Intero più grande	$2^{31} - 1$ per `int`	$2^{31} - 1$ per `int`	nessun limite
Operatori logici	`&&`, `||`, `!`	`&&`, `||`, `!`	`and`, `or`, `not`

Tabella di confronto (3/3)

	C++	Java	Python
Collezioni	array, liberati con `delete[]`	array (`a.length`) e `ArrayList`, liberati automaticamente	liste (`len(a)`), liberate automaticamente
Indice fuori intervallo	non controllato	eccezione	eccezione
Più risultati	parametri per riferimento	restituzione di un array	restituzione di una tupla
Punto di ingresso	funzione `main`	metodo `main` di una classe	livello più esterno del file

Riepilogo

- `Java` ha in comune con `C++` i tipi di dato dichiarati, i blocchi fra parentesi graffe, il "`;`", la divisione intera e l'`int` a 32 bit
- `Java` ha in comune con `Python` gli indici controllati e il rilascio automatico della memoria
- Solo `Java` richiede una classe intorno a ogni programma, e restituisce più risultati in un array

Riepilogo (1/2)

Punti chiave del capitolo

- Un programma `Java` è costituito da classi; è compilato in bytecode con `javac` ed eseguito dalla JVM con `java`, a partire dal metodo `public static void main(String[] args)`
- Ogni variabile è dichiarata con il suo tipo di dato, che è controllato dal compilatore; il tipo di dato `int` ha 32 bit, e la divisione fra due interi è la divisione intera
- Le istruzioni condizionali (`if`, `else if`, `else`, `?:`) e i cicli (`while`, `for`, e il `for` esteso) hanno la stessa sintassi di `C++`, ma una condizione deve essere di tipo `boolean`

Riepilogo (2/2)

Punti chiave del capitolo

- Gli array di `Java` si allocano con `new`, conoscono la loro dimensione `a.length`, sono indicizzati da 0 a $n-1$ e sono liberati automaticamente; la classe `ArrayList` fornisce liste di dimensione variabile

- I parametri sono passati per valore; un metodo che riceve un array può modificarne gli elementi, e più risultati sono restituiti insieme in un array